

Per-session color plan

Per-session color (name:color[:params]) Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (– []) syntax for tracking.

Goal: Let the operator choose an explicit, persisted, per-session tmux color via `tmx new -s name:color[:ignored]`, validated (tmux names + #hex), applied to all 4 "green" surfaces, and re-used on bare-name re-runs.

Architecture: Approach A from the spec — parse the `-s` value in the bash wrapper (pure helpers in a sourced lib), validate + persist in the Go `tmx-state-bin` (`set-color/get-color`, schema 1→2 with an additive color field), and apply via a new `_apply_color` that mirrors the existing `_apply_host_color`. Precedence: inline > persisted > hostname > default-green. The macOS bridge is unchanged (forwards `-s` verbatim), so §11.4.81 parity is free.

Tech Stack: Bash (POSIX-portable helpers + bash dispatcher), Go 1.21+ (state binary), tmux 3.6a options API, pandoc/weasyprint (doc exports), the repo's `commit_all.sh` + `run_all.sh` + meta-test harness.

Spec: <docs/superpowers/specs/2026-06-18-session-color-design.md>

Global Constraints

- **Anti-bluff (§11.4/§102):** every runtime PASS reads live server state (`tmux -L <sock> show-options -gv ...`), never an exit code alone. Tests drive the real operator path (`$WRAPPER ...`), never a hand-spawned equivalent.
- **No guessing (§11.4.6):** invalid color → hard error + non-zero exit + **no session created**. The bash and Go color-accept sets are byte-identical copies of one canonical list.
- **Cross-OS (§11.4.81):** no Darwin-specific branch; the bridge forwards `-s` verbatim and the VM-side wrapper does all parsing.
- **No force-push (§11.4.113); commit only via `commit_all.sh`.** Exports (html/pdf/docx) regenerated for any tracked `.md` doc touched (§11.4.65).

- **Parseability (§11.4.67):** every edited shell script passes `sh -n (helpers) / bash -n (dispatcher)`.
- **Determinism (§11.4.50):** runtime tests reproducible; same inputs → same status-style readback.
- **Versioning (§11.4.151):** release tag prefixed `tmux-<version>` (prefix from `HELIX_RELEASE_PREFIX` or lowercased root dir name).

File Structure

File	Responsibility	Status
<code>scripts/tmx-color-lib.sh</code>	Pure bash helpers: _parse_session_value, _color_valid, CANON_COLOR_NAMES. Sourced by the wrapper + by unit tests.	create
<code>scripts/tmx.template</code>	Generated wrapper: source the lib, call _resolve_color, _apply_color; wire into new/attach.	modify
<code>scripts/tmx-state/state.go</code>	Session.Color field, SchemaVersion 1→2, validColor() Go twin.	modify
<code>scripts/tmx-state/color.go</code>	Go validColor() + CANON_COLOR_NAMES (byte-twin of the bash list).	create
<code>scripts/tmx-state/color_test.go</code>	Prove Go + bash lists agree on a probe set.	create
<code>scripts/tmx-state/main.go</code>	set-color/get-color subcommands, list 4th column, version bump.	modify
<code>scripts/tmx-state/state_test.go / main_test.go</code>	Round-trip Color, schema compat, subcommand contracts.	modify
<code>scripts/tests/63_session_color.sh</code>	Runtime operator-path suite (8 sub-tests T1–T8).	create
<code>scripts/tests/ 64_session_color_parse_unit.sh</code>	Pure-fn unit tests for the bash lib (no tmux needed).	create
<code>scripts/tests/ meta_test_false_positive_proof.sh</code>	Add M23/M24 paired mutations for _apply_color + set- color.	modify
<code>scripts/challenges/tmux.yaml</code>	Add TMUX-CH-XX color Challenge.	modify
<code>docs/guide/README.md, CLAUDE.md, Fixed.md, CHANGELOG.md, VERSION</code>	Docs + version.	modify
	This plan + exports.	create

File	Responsibility	Status
docs/superpowers/plans/ 2026-06-18-session-color. {md,html,pdf,docx}		

Task 1: Go state binary — color.go validator (TDD)

Files:

- Create: scripts/tmx-state/color.go
- Test: scripts/tmx-state/color_test.go

Interfaces:

- Produces: func validColor(s string) bool and
var CanonColorNames = [...]string (Go), byte-identical to the bash
CANON_COLOR_NAMES list added in Task 6.



Step 1: Write the failing test

Create scripts/tmx-state/color_test.go:

```
package main
```

```
import "testing"
```

```
func TestValidColor(t *testing.T) {
    good := []string{
        "red", "Red", "RED", // case-insensitive names
        "green", "yellow", "blue", "magenta", "cyan", "white",
        "black",
        "brightred", "brightcyan", "default", "terminal",
        "colour0", "colour255", "colour39", "color160", "Color7",
        "#3b82f6", "#FFF", "#f0a", "#000000",
    }
    for _, c := range good {
        if !validColor(c) {
            t.Errorf("validColor(%q) = false, want true", c)
        }
    }
    bad := []string{
        "", "purple", "colour256", "colour-1", "colour",
        "color1234",
        "#12", "#12345", "#GGG", "3b82f6", "red ", " red",
    }
    for _, c := range bad {
        if validColor(c) {
            t.Errorf("validColor(%q) = true, want false", c)
        }
    }
}
```

```

    }
}

// TestCanonColorNamesBashTwin asserts the Go list is byte-
// identical to the
// bash lib's list. The exact same space-separated string appears
// in
// scripts/tmx-color-lib.sh::CANON_COLOR_NAMES. A divergence is a
// $11.4.6
// guessing surface. (The bash side mirrors this in test 64.)
func TestCanonColorNamesBashTwin(t *testing.T) {
    want := "red green yellow blue magenta cyan white black
    brightred brightgreen brightyellow brightblue
    brightmagenta brightcyan brightwhite default terminal"
    got := ""
    for i, n := range CanonColorNames {
        if i > 0 {
            got += " "
        }
        got += n
    }
    if got != want {
        t.Errorf("CanonColorNames drift:\n got: %q\nwant: %q",
            got, want)
    }
}

```



Step 2: Run test to verify it fails

Run: `cd scripts/tmx-state && go test -run 'TestValidColor|TestCanonColorNamesBashTwin' -v ./...` Expected: build FAIL / undefined: validColor.



Step 3: Write minimal implementation

Create `scripts/tmx-state/color.go`:

```

// Package main – color token validation (Go twin of the bash
// lib).
//
// The accepted set is the SINGLE source of truth shared with the
// bash
// helper scripts/tmx-color-lib.sh (CANON_COLOR_NAMES).
// color_test.go +
// scripts/tests/64_session_color_parse_unit.sh assert both sides
// agree on
// a probe set, so persistence (Go) and CLI parsing (bash) can
// never
// disagree on what a valid color is (§11.4.6).

package main

```

```

import (
    "regexp"
    "strings"
)

// CanonColorNames – byte-identical twin of bash
//                      CANON_COLOR_NAMES.
// Keep in lockstep with scripts/tmux-color-lib.sh.
var CanonColorNames = []string{
    "red", "green", "yellow", "blue", "magenta", "cyan", "white",
    "black",
    "brightred", "brightgreen", "brightyellow", "brightblue",
    "brightmagenta",
    "brightcyan", "brightwhite", "default", "terminal",
}

var (
    reColourIdx = regexp.MustCompile(`^(?i)colou?r([0-9]{1,3})$`)
    reHex       = regexp.MustCompile(`^#[0-9A-Fa-f]{3}([0-9A-Fa-f]{3})?$`)
)

// validColor reports whether s is a tmux-acceptable color token:
//   - a canonical name (case-insensitive);
//   - colourNNN / colorNNN with N in 0..255;
//   - #RGB or #RRGGBB hex.
func validColor(s string) bool {
    if s == "" {
        return false
    }
    low := strings.ToLower(s)
    for _, n := range CanonColorNames {
        if low == n {
            return true
        }
    }
    if m := reColourIdx.FindStringSubmatch(s); m != nil {
        // m[1] is 1-3 digits; value range check.
        var v int
        for _, ch := range m[1] {
            v = v*10 + int(ch-'0')
        }
        return v >= 0 && v <= 255
    }
    return reHex.MatchString(s)
}

```



Step 4: Run test to verify it passes

Run: `cd scripts/tmx-state && go test -run 'TestValidColor|TestCanonColorNamesBashTwin' -v ./... Expected: PASS (both tests).`



Step 5: Commit

```
cd /Volumes/T7/Projects/tmux
git add scripts/tmx-state/color.go scripts/tmx-state/color_test.go
bash commit_all.sh "feat(tmx-state §102): validColor +
    CanonColorNames (Go twin of bash lib) - TDD"
```

Co-Authored-By: Claude Opus 4.8 (1M context)
<noreply@anthropic.com>

Task 2: Go state binary — Color field + schema bump (TDD)

Files:

- Modify: `scripts/tmx-state/state.go`
- Test: `scripts/tmx-state/state_test.go`

Interfaces:

- Produces: `Session.Color` string (json color, omit empty);
SchemaVersion == 2. Additive — schema-1 files load with `Color=""`.



Step 1: Write the failing test

Append to `scripts/tmx-state/state_test.go`:

```
func TestSessionColorRoundTrip(t *testing.T) {
    tmp := filepath.Join(t.TempDir(), "state.json")
    st := newEmptyState()
    st.SchemaVersion = SchemaVersion // current (2)
    st.Sessions["work"] = Session{
        LastPwd: "/tmp", LastSeenUnix: 1, CreatedUnix: 1, Color:
        "red",
    }
    if err := saveState(tmp, st); err != nil {
        t.Fatalf("save: %v", err)
    }
    got, err := loadState(tmp)
    if err != nil {
        t.Fatalf("load: %v", err)
    }
    if got.Sessions["work"].Color != "red" {
        t.Errorf("Color round-trip = %q, want red",
            got.Sessions["work"].Color)
    }
}
```

```

    if got.SchemaVersion != 2 {
        t.Errorf("SchemaVersion = %d, want 2", got.SchemaVersion)
    }
}

// TestSchema1FileLoadsAsEmptyColor proves an old (schema-1) file
// with no
// color field loads cleanly with Color=="" (forward compat -
// additive
// omitempty field).
func TestSchema1FileLoadsAsEmptyColor(t *testing.T) {
    tmp := filepath.Join(t.TempDir(), "state.json")
    // Hand-write a schema-1 file with no color field.
    old := []byte(`{"schema_version":1,"sessions":{"work":
        {"last_pwd":"/x","last_seen_unix":1,"created_unix":1}}}`
        + "\n")
    if err := os.WriteFile(tmp, old, 0o600); err != nil {
        t.Fatal(err)
    }
    got, err := loadState(tmp)
    if err != nil {
        t.Fatalf("load schema-1: %v", err)
    }
    if got.Sessions["work"].Color != "" {
        t.Errorf("legacy Color = %q, want empty",
            got.Sessions["work"].Color)
    }
}

```

(Add "path/filepath" to the imports of state_test.go if not present.)



Step 2: Run test to verify it fails

Run: `cd scripts/tmx-state && go test -run 'TestSessionColorRoundTrip|TestSchema1FileLoadsAsEmptyColor' -v ./...` Expected: FAIL (Session.Color undefined) / compile error on SchemaVersion==2.



Step 3: Write minimal implementation

In `scripts/tmx-state/state.go`:

Change SchemaVersion:

```
const SchemaVersion = 2
```

Add the field to Session:

```

type Session struct {
    LastPwd      string `json:"last_pwd"`
    LastSeenUnix int64  `json:"last_seen_unix"`
    CreatedUnix  int64  `json:"created_unix"`
}

```

```

Host      string `json:"host,omitempty"`
Color     string `json:"color,omitempty"`
}

```



Step 4: Run test to verify it passes

Run: `cd scripts/tmx-state && go test -run 'TestSessionColorRoundTrip|TestSchema1FileLoadsAsEmptyColor' -v ./...` Expected: PASS. Also run the full package to confirm no regression: `cd scripts/tmx-state && go test -count=1 ./... → all PASS.`



Step 5: Commit

```

cd /Volumes/T7/Projects/tmux
git add scripts/tmx-state/state.go scripts/tmx-state/state_test.go
bash commit_all.sh "feat(tmx-state §11.4.93): Session.Color +
    schema 1→2 (additive, forward+back compat)

```

Co-Authored-By: Claude Opus 4.8 (1M context)
<noreply@anthropic.com>"

Task 3: Go state binary — set-color / get-color subcommands (TDD)

Files:

- Modify: `scripts/tmx-state/main.go`
- Test: `scripts/tmx-state/main_test.go`

Interfaces:

- Produces: run dispatches `set-color <session> <color>` (exit 0 on success, 1 on invalid color or IO) and `get-color <session>` (prints color no-newline + exit 0; empty + exit 1 if none).



Step 1: Write the failing test

Append to `scripts/tmx-state/main_test.go`:

```

func TestSetColorAndGetColor(t *testing.T) {
    tmp := filepath.Join(t.TempDir(), "state.json")
    t.Setenv("TMX_STATE_FILE", tmp)

    var buf bytes.Buffer
    var stderr bytes.Buffer
    // set-color work red
    if rc := run([]string{"set-color", "work", "red"}, &buf,
        &stderr); rc != 0 {

```



```

        t.Fatalf("set-color rc=%d stderr=%s", rc, stderr.String())
    }
    // get-color work → "red"
    buf.Reset(); stderr.Reset()
    if rc := run([]string{"get-color", "work"}, &buf, &stderr);
        rc != 0 {
        t.Fatalf("get-color rc=%d", rc)
    }
    if buf.String() != "red" {
        t.Errorf("get-color = %q, want red", buf.String())
    }
    // get-color unknown → exit 1, empty
    buf.Reset()
    if rc := run([]string{"get-color", "nope"}, &buf, &stderr);
        rc != 1 {
        t.Errorf("get-color unknown rc=%d, want 1", rc)
    }
    if buf.String() != "" {
        t.Errorf("get-color unknown = %q, want empty",
            buf.String())
    }
}

func TestSetColorInvalid(t *testing.T) {
    tmp := filepath.Join(t.TempDir(), "state.json")
    t.Setenv("TMX_STATE_FILE", tmp)
    var stdout, stderr bytes.Buffer
    if rc := run([]string{"set-color", "work", "notacolor"},
        &stdout, &stderr); rc != 1 {
        t.Errorf("set-color invalid rc=%d, want 1", rc)
    }
}

// TestSetColorPreservesSiblingFields – set-color must not clobber
// last_pwd.
func TestSetColorPreservesSiblingFields(t *testing.T) {
    tmp := filepath.Join(t.TempDir(), "state.json")
    t.Setenv("TMX_STATE_FILE", tmp)
    var o, e bytes.Buffer
    run([]string{"record", "work", "/tmp"}, &o, &e)
    o.Reset(); e.Reset()
    run([]string{"set-color", "work", "blue"}, &o, &e)
    // reload + check last_pwd intact
    st, err := loadState(tmp)
    if err != nil {
        t.Fatal(err)
    }
    if st.Sessions["work"].LastPwd != "/tmp" {
        t.Errorf("last_pwd clobbered = %q",
            st.Sessions["work"].LastPwd)
    }
    if st.Sessions["work"].Color != "blue" {
        t.Errorf("color = %q, want blue",
            st.Sessions["work"].Color)
    }
}

```

```
}
}
```

(Ensure "bytes" and "path/filepath" are imported in main_test.go.)



Step 2: Run test to verify it fails

Run: `cd scripts/tmx-state && go test -run 'TestSetColor|TestGetColor' -v ./...` Expected: FAIL — run has no set-color/get-color cases (falls to unknown subcommand).



Step 3: Write minimal implementation

In `scripts/tmx-state/main.go`, add to the usage text (after the forget line):

```
tmx-state set-color <session> <color>
tmx-state get-color <session>
```

Add two cases to the switch sub in run:

```
case "set-color":
    return cmdSetColor(rest, stderr)
case "get-color":
    return cmdGetColor(rest, stdout, stderr)
```

Add the two handlers (after `cmdForget`):

```
// cmdSetColor: tmx-state set-color <session> <color>
// Validates <color> with validColor ($color.go), then persists it
// under
// the existing lock, preserving all sibling fields. Exit 1 on
// invalid
// color or IO error.
func cmdSetColor(args []string, stderr io.Writer) int {
    fs := flag.NewFlagSet("set-color", flag.ContinueOnError)
    fs.SetOutput(stderr)
    if err := fs.Parse(args); err != nil {
        return 1
    }
    if fs.NArg() != 2 {
        fmt.Fprintln(stderr, "tmx-state set-color: expected
        <session> <color>")
        return 1
    }
    session := fs.Arg(0)
    color := fs.Arg(1)
    if session == "" {
        fmt.Fprintln(stderr, "tmx-state set-color: empty session
        name")
        return 1
    }
    if !validColor(color) {
```

```

        fmt.Fprintf(stderr, "tmx-state set-color: invalid color
        %q\n", color)
        return 1
    }
    path, err := statePath()
    if err != nil {
        fmt.Fprintf(stderr, "tmx-state set-color: %v\n", err)
        return 1
    }
    var rc int
    werr := withStateLock(path, func() error {
        st, lerr := loadState(path)
        if lerr != nil && !errors.Is(lerr, errStateRebuilt) {
            fmt.Fprintf(stderr, "tmx-state set-color: load:
            %v\n", lerr)
            rc = 1
            return nil
        }
        if errors.Is(lerr, errStateRebuilt) {
            fmt.Fprintln(stderr, "tmx-state: notice: state file
            was corrupt, rebuilt")
        }
        existing, had := st.Sessions[session]
        now := time.Now().Unix()
        s := existing
        if !had {
            s = Session{CreatedUnix: now}
        }
        s.Color = color
        s.LastSeenUnix = now
        st.Sessions[session] = s
        if err := saveStateUnlocked(path, st); err != nil {
            fmt.Fprintf(stderr, "tmx-state set-color: save:
            %v\n", err)
            rc = 1
            return nil
        }
        return nil
    })
    if werr != nil {
        fmt.Fprintf(stderr, "tmx-state set-color: %v\n", werr)
        return 1
    }
    return rc
}

// cmdGetColor: tmx-state get-color <session>
// Prints the persisted color (no newline) + exit 0; empty + exit
// 1 if
// none/unset. Mirrors recall's contract.
func cmdGetColor(args []string, stdout, stderr io.Writer) int {
    fs := flag.NewFlagSet("get-color", flag.ContinueOnError)
    fs.SetOutput(stderr)

```

```

    if err := fs.Parse(args); err != nil {
        return 1
    }
    if fs.NArg() != 1 {
        fmt.Fprintln(stderr, "tmx-state get-color: expected
        <session>")
        return 1
    }
    session := fs.Arg(0)
    path, err := statePath()
    if err != nil {
        fmt.Fprintf(stderr, "tmx-state get-color: %v\n", err)
        return 1
    }
    st, lerr := loadState(path)
    if errors.Is(lerr, errStateRebuilt) {
        return 1
    }
    if lerr != nil {
        fmt.Fprintf(stderr, "tmx-state get-color: %v\n", lerr)
        return 1
    }
    sess, ok := st.Sessions[session]
    if !ok || sess.Color == "" {
        return 1
    }
    fmt.Fprint(stdout, sess.Color)
    return 0
}

```



Step 4: Run test to verify it passes

Run: `cd scripts/tmx-state && go test -count=1 ./... Expected: all PASS (existing + new).`



Step 5: Commit

```

cd /Volumes/T7/Projects/tmux
git add scripts/tmx-state/main.go scripts/tmx-state/main_test.go
bash commit_all.sh "feat(tmx-state §102): set-color/get-color
subcommands (validate+persist under lock)

```

Co-Authored-By: Claude Opus 4.8 (1M context)
 <noreply@anthropic.com>

Task 4: Go state binary — list 4th column + version bump

Files:

- Modify: scripts/tmx-state/main.go

Interfaces:

- Produces: list prints SESSION\tLAST_PWD\tLAST_SEEN_UNIX\tCOLOR;
Version = "tmx-state v1.1.0".



Step 1: Update list + Version

In scripts/tmx-state/main.go:

Change the Version const:

```
const Version = "tmx-state v1.1.0"
```

In cmdList, change the header/comment and the print line:

```
// Prints SESSION\tLAST_PWD\tLAST_SEEN_UNIX\tCOLOR lines, sorted  
by session name.
```

```
...
```

```
fmt.Fprintf(stdout, "%s\t%s\t%d\t%s\n", n, s.LastPwd,  
s.LastSeenUnix, s.Color)
```



Step 2: Run the full Go test suite

Run: `cd scripts/tmx-state && go test -count=1 ./...` Expected: all PASS. If an existing cmdList test asserts the old 3-column shape, update it to the 4-column shape in the same edit (reconcile per §11.4.120 — the gate asserted the OLD behaviour because the feature legitimately changed it; rewrite to assert the NEW behaviour). Find it with: `grep -rn 'list' scripts/tmx-state/main_test.go`



Step 3: Build the binary

Run: `cd /Volumes/T7/Projects/tmux && (cd scripts/tmx-state && go build -o ../tmx-state-bin .) && scripts/tmx-state-bin version`
Expected: prints tmx-state v1.1.0.



Step 4: Commit

```
cd /Volumes/T7/Projects/tmux  
git add scripts/tmx-state/main.go scripts/tmx-state-bin  
bash commit_all.sh "feat(tmx-state): list 4th column (COLOR) +  
version v1.1.0 + rebuild binary"
```

Task 5: Bash pure helpers — tmx-color-lib.sh (TDD)

Files:

- Create: scripts/tmx-color-lib.sh
- Create test: scripts/tests/64_session_color_parse_unit.sh

Interfaces:

- Produces (bash, global side-effects via stdout/stderr only on `_color_valid`):
 - `_parse_session_value <raw>` → sets `PARSED_NAME` + `PARSED_COLOR` (globals).
 - `_color_valid <token>` → return 0/1.
 - `CANON_COLOR_NAMES` — space-separated string, byte-twin of Go `CanonColorNames`.



Step 1: Write the failing test

Create scripts/tests/64_session_color_parse_unit.sh:

```
#!/usr/bin/env bash
# Test 64 – pure-fn unit tests for the session-color bash lib.
# No tmux / no wrapper needed; sources the lib directly.
set -uo pipefail
REPO_ROOT="$(cd "$(dirname "$0")/../../.." && pwd)"
LIB="$REPO_ROOT/scripts/tmx-color-lib.sh"

echo "— Test 64: session-color pure-fn unit tests —"
PASS=0; FAIL=0
_pass() { echo "PASS: $*"; PASS=$((PASS+1)); }
_fail() { echo "FAIL: $*"; FAIL=$((FAIL+1)); }

[ -f "$LIB" ] || { echo "FAIL: lib missing: $LIB"; exit 1; }
# shellcheck disable=SC1090
source "$LIB"

# --- _parse_session_value table (matches spec §4) ---
chk_parse() { # chk_parse <input> <want_name> <want_color>
    _parse_session_value "$1"
    if [ "$PARSED_NAME" = "$2" ] && [ "$PARSED_COLOR" = "$3" ];
    then
        _pass "parse '$1' → name='$2' color='$3'"
    else
        _fail "parse '$1' → name='$PARSED_NAME'
            color='$PARSED_COLOR' (want '$2'/'$3')"
```

```

    fi
}
chk_parse "work"          "work"  ""
chk_parse "work:red"      "work"  "red"
chk_parse "deploy:#3b82f6" "deploy" "#3b82f6"
chk_parse 'a\b:cyan'      "a:b"   "cyan"
chk_parse "work:red:x:y"  "work"  "red"
chk_parse "work:"         "work"  ""

# --- _color_valid table ---
chk_cv() { # chk_cv <token> <want 0|1>
    if _color_valid "$1"; then _color_ok=0; else _color_ok=1; fi
    if [ "$_color_ok" = "$2" ]; then _pass "valid '$1' → $2"
    else _fail "valid '$1' → $_color_ok (want $2)"; fi
}
chk_cv "red"          0
chk_cv "RED"          0
chk_cv "colour160"    0
chk_cv "color39"      0
chk_cv "#3b82f6"      0
chk_cv "#f0a"         0
chk_cv "purple"       1
chk_cv "colour256"    1
chk_cv "#12"          1
chk_cv ""             1

# --- bash↔Go list parity (mirrors Go TestCanonColorNamesBashTwin)
---
WANT="red green yellow blue magenta cyan white black brightred
      brightgreen brightyellow brightblue brightmagenta
      brightcyan brightwhite default terminal"
if [ "$CANON_COLOR_NAMES" = "$WANT" ]; then
    _pass "bash CANON_COLOR_NAMES matches Go twin"
else
    _fail "bash CANON_COLOR_NAMES drift: '$CANON_COLOR_NAMES'"
fi

echo "— Test 64 result: PASS=$PASS FAIL=$FAIL —"
[ "$FAIL" -eq 0 ]

```



Step 2: Run test to verify it fails

Run: `cd /Volumes/T7/Projects/tmux && bash scripts/tests/64_session_color_parse_unit.sh` Expected: FAIL — lib missing.



Step 3: Write minimal implementation

Create `scripts/tmx-color-lib.sh`:

```

# tmx-color-lib.sh – pure helpers for per-session color parsing/
# validation.
#
# Sourced by the generated tmx wrapper AND by unit test
# scripts/tests/64_session_color_parse_unit.sh. Contains NO side
# effects,
# NO tmux calls, NO state writes – only:
#   _parse_session_value <raw>    sets globals PARSED_NAME +
#     PARSED_COLOR
#   _color_valid <token>          return 0 iff token is a valid
#     tmux color
#   CANON_COLOR_NAMES             space-list, byte-twin of Go
#     CanonColorNames
#
# Canonical name set – keep byte-identical to scripts/tmx-state/
#   color.go
# (CanonColorNames). A divergence is a §11.4.6 guessing surface;
#   both sides
# are cross-checked by Go TestCanonColorNamesBashTwin + test 64.
CANON_COLOR_NAMES="red green yellow blue magenta cyan white black
  brightred brightgreen brightyellow brightblue
  brightmagenta brightcyan brightwhite default terminal"

# _parse_session_value <raw>
# Split on unescaped ':'. Escapes (\:) honored ONLY in field 0
#   (the name).
# Sets: PARSED_NAME (raw, pre-sanitise), PARSED_COLOR ("" if
#   none).
# Pure; emits nothing to stdout/stderr.
_parse_session_value() {
  PARSED_NAME=""
  PARSED_COLOR=""
  local raw="${1-}"
  local i=0 ch field=0 name="" color="" rest=""
  local len=${#raw}
  while [ "$i" -lt "$len" ]; do
    ch="${raw:$i:1}"
    # Escape handling only in field 0.
    if [ "$field" -eq 0 ] && [ "$ch" = '\' ]; then
      local nxt="${raw:$((i+1)):1}"
      if [ "$nxt" = ":" ]; then
        name="${name}:" # unescape \: → literal :
        i=$((i+2))
        continue
      fi
      # A backslash not before ':' is a literal backslash.
      name="${name}\\\"
      i=$((i+1))
      continue
    fi
    if [ "$ch" = ":" ]; then
      field=$((field+1))
      i=$((i+1))
      continue
    fi
  done
}

```



```

    fi
    if [ "$field" -eq 0 ]; then
        name="${name}${ch}"
    elif [ "$field" -eq 1 ]; then
        color="${color}${ch}"
    else
        rest="${rest}${ch}"
    fi
    i=$((i+1))
done
PARSED_NAME="$name"
PARSED_COLOR="$color"
# rest is intentionally discarded (forward-compatible;
# decision #5).
return 0
}

# _color_valid <token>
# Return 0 iff token is a valid tmux color (name / colourNNN /
# hex).
_color_valid() {
    local t="${1-}"
    [ -n "$t" ] || return 1
    # Canonical names, case-insensitive.
    local low
    low=$(printf '%s' "$t" | tr '[:upper:]' '[:lower:]')
    local n
    for n in $CANON_COLOR_NAMES; do
        [ "$low" = "$n" ] && return 0
    done
    # colourNNN / colorNNN, 0..255.
    case "$t" in
        [Cc][Oo][Ll][Oo][Uu][Rr][0-9] | [Cc][Oo][Ll][Oo][Uu][Rr]
        [0-9][0-9] | [Cc][Oo][Ll][Oo][Uu][Rr][0-9][0-9][0-9])
            local num="${t#colou}" ; num="${num#colo}" ; num="$
            {num#colour}" ; num="${num#color}"
            # strip any leading non-digit noise already excluded
            by case
            case "$num" in ''|*![0-9]*) return 1 ;; esac
            [ "$num" -ge 0 ] 2>/dev/null && [ "$num" -le 255 ] 2>/
            dev/null && return 0
            return 1
            ;;
        [Cc][Oo][Ll][Oo][Rr][0-9] | [Cc][Oo][Ll][Oo][Rr][0-9][0-9] |
        [Cc][Oo][Ll][Oo][Rr][0-9][0-9][0-9])
            local num="${t#color}"
            case "$num" in ''|*![0-9]*) return 1 ;; esac
            [ "$num" -ge 0 ] 2>/dev/null && [ "$num" -le 255 ] 2>/
            dev/null && return 0
            return 1
            ;;
    esac
    # #RGB or #RRGGBB hex.
    case "$t" in

```

```

        \#[0-9Aa-Ff] [0-9Aa-Ff] [0-9Aa-Ff]) return 0 ;;
        \#[0-9Aa-Ff] [0-9Aa-Ff] [0-9Aa-Ff] [0-9Aa-Ff] [0-9Aa-Ff]
        [0-9Aa-Ff]) return 0 ;;
    esac
    return 1
}

```



Step 4: Run test to verify it passes + parse-check

Run:

```

cd /Volumes/T7/Projects/tmux && sh -n scripts/tmx-color-lib.sh &&
bash scripts/tests/64_session_color_parse_unit.sh Expected: PASS=26
FAIL=0 (all table rows + parity), exit 0.

```



Step 5: Commit

```

cd /Volumes/T7/Projects/tmux
git add scripts/tmx-color-lib.sh scripts/tests/
    64_session_color_parse_unit.sh
bash commit_all.sh "feat(tmx §102): tmx-color-lib.sh pure helpers
    (_parse_session_value/_color_valid) + unit test 64

```

Co-Authored-By: Claude Opus 4.8 (1M context)
 <noreply@anthropic.com>"

Task 6: Bash wrapper — `_resolve_color` + `_apply_color` + wiring

Files:

- Modify: `scripts/tmx.template`

Interfaces:

- Consumes: `_parse_session_value`, `_color_valid` (Task 5); `tmx-state-bin set-color/get-color` (Tasks 3–4).
- Produces: wrapper resolves `EFFECTIVE_COLOR` and applies it via `_apply_color`, falling back to `_apply_host_color` when empty.



Step 1: Source the lib + add helpers

Near the top of `scripts/tmx.template`, after the `TMX_DIR=...` line (around line 37), add:

```

# Pure color helpers (sourced so they are unit-testable in
    isolation).
if [ -f "$TMX_DIR/tmx-color-lib.sh" ]; then
    # shellcheck disable=SC1091

```

```

    . "$TMX_DIR/tmx-color-lib.sh"
fi

```

Just above the `_apply_host_color` definition (around line 132), add two new helpers:

```

# Resolve the effective color for a session.
# Args: <name> <inline_candidate>
# - if inline candidate non-empty: validate; on invalid, return
#   code 5
#   (caller fails-fast, no session created); on valid, best-effort
#   persist
#   via tmx-state-bin (non-fatal) and print it.
# - else: read persisted color (non-fatal if state binary absent);
#   print it.
# - else: print nothing (caller falls back to hostname color).
# Non-fatal posture preserves the wrapper's "tmx-never-breaks"
# invariant.
_resolve_color() {
    local name="$1" cand="${2-}"
    if [ -n "$cand" ]; then
        if ! _color_valid "$cand"; then
            printf 'tmx: invalid color %q (use a tmux name,
            colour0-255, or #hex)\n' "$cand" >&2
            return 5
        fi
        if [ -x "$TMX_DIR/tmx-state-bin" ]; then
            "$TMX_DIR/tmx-state-bin" set-color "$name" "$cand" >/
            dev/null 2>&1 || true
        fi
        printf '%s' "$cand"
        return 0
    fi
    if [ -x "$TMX_DIR/tmx-state-bin" ]; then
        "$TMX_DIR/tmx-state-bin" get-color "$name" 2>/dev/null ||
        true
    fi
    return 0
}

# Apply an explicit color to the 4 "green" surfaces (mirrors
# _apply_host_color
# minus the hostname derivation). Best-effort + silent on failure.
_apply_color() {
    local sock_label="$1" color="$2"
    "$TMUX_BIN" -L "$sock_label" set -g status-style
    "bg=$color" 2>/dev/null || true
    "$TMUX_BIN" -L "$sock_label" set -g pane-active-border-style
    "fg=$color" 2>/dev/null || true
    "$TMUX_BIN" -L "$sock_label" set -g clock-mode-colour
    "$color" 2>/dev/null || true
    "$TMUX_BIN" -L "$sock_LABEL" set -g window-status-current-
    style "bg=$color,fg=black" 2>/dev/null || true
}

```

⚠ **Self-review catch (fix the typo before saving):** the last line uses `$sock_LABEL` (uppercase) — that variable doesn't exist. It MUST be `$sock_label`. Use this corrected line:

```
"$TMUX_BIN" -L "$sock_label" set -g window-status-current-  
style "bg=$color,fg=black" 2>/dev/null || true
```



Step 2: Wire `_parse_session_value` into NAME resolution

In `scripts/tmx.template`, replace this block (around lines 244–251):

```
NAME=""  
[ -n "$SESSION_ARG" ] && NAME="$SESSION_ARG"  
[ -z "$NAME" ] && [ -n "$TARGET_ARG" ] && NAME="$TARGET_ARG"  
[ -z "$NAME" ] && NAME="default"
```

```
SAFE_NAME=$(_sanitise "$NAME")  
SOCK_LABEL="${TMX_SOCK_PREFIX}${SAFE_NAME}"
```

with:

```
RAW_NAME=""  
[ -n "$SESSION_ARG" ] && RAW_NAME="$SESSION_ARG"  
[ -z "$RAW_NAME" ] && [ -n "$TARGET_ARG" ] &&  
    RAW_NAME="$TARGET_ARG"  
[ -z "$RAW_NAME" ] && RAW_NAME="default"  
  
# $!02 per-session color: split "name:color[:ignored]" on  
# unescaped ':'.  
if command -v _parse_session_value >/dev/null 2>&1; then  
    _parse_session_value "$RAW_NAME"  
else  
    PARSED_NAME="$RAW_NAME"; PARSED_COLOR=""  
fi  
NAME=$(_sanitise "$PARSED_NAME")  
# Resolve effective color (inline > persisted > "" → hostname  
# fallback).  
# Invalid inline color → exit 5 BEFORE any socket/scope/session is  
# created.  
EFFECTIVE_COLOR=""  
if command -v _resolve_color >/dev/null 2>&1; then  
    EFFECTIVE_COLOR=$(_resolve_color "$NAME" "$PARSED_COLOR") ||  
        rc=$?  
    rc=${rc:-0}  
    if [ "${rc}" -eq 5 ]; then  
        exit 5  
    fi  
fi  
SAFE_NAME="$NAME"  
SOCK_LABEL="${TMX_SOCK_PREFIX}${SAFE_NAME}"
```



Step 3: Apply the color in the new branch

In the `new|new-session|start-server|""` case, replace the line `_apply_host_color "$SOCK_LABEL"` (around line 408) with:

```
if [ -n "$EFFECTIVE_COLOR" ]; then
    _apply_color "$SOCK_LABEL" "$EFFECTIVE_COLOR"
else
    _apply_host_color "$SOCK_LABEL"
fi
```



Step 4: Re-apply on attach

In the `attach|attach-session|a` case, after the existing source-file reload (around line 467), add before the `exec`:

```
if [ -n "${EFFECTIVE_COLOR:-}" ]; then
    _apply_color "$SOCK_LABEL" "$EFFECTIVE_COLOR"
fi
```



Step 5: Parse-check + regenerate the wrapper

Run:

```
cd /Volumes/T7/Projects/tmux
bash -n scripts/tmx.template
sh -n scripts/tmux.conf.template
# Regenerate the gitignored dispatcher so the live wrapper carries
# the change.
TMX_FORCE_REGEN=1 bash scripts/setup.sh --build-only 2>&1 | tail
-5
```

Expected: `bash -n/sh -n` silent (no errors); setup regenerates `scripts/tmx`.



Step 6: Commit

```
cd /Volumes/T7/Projects/tmux
git add scripts/tmx.template
bash commit_all.sh "feat(tmx §102): _resolve_color/_apply_color +
    name:color wiring in new/attach branches"
```

Co-Authored-By: Claude Opus 4.8 (1M context)
<noreply@anthropic.com>

Task 7: Runtime operator-path test suite (test 63)

Files:

- Create: `scripts/tests/63_session_color.sh`

Interfaces:

- Consumes: built \$WRAPPER, \$TMUX_BIN, \$TMX_DIR/tmx-state-bin, scripts/tmx-color-lib.sh. Drives the real operator path.



Step 1: Write the test

Create scripts/tests/63_session_color.sh (mirror the structure of test 17 — operator-path + captured-evidence + cleanup trap):

```
#!/usr/bin/env bash
# Test 63 – per-session color (operator-path, anti-bluff).
#
# $102: drives the SAME entry point an end user invokes – `tmx new
#       -s ...`.
# $11.4.2/$11.4.5: every PASS reads LIVE server state via show-
#                  options
# (status-style / pane-active-border-style / clock-mode-colour /
# window-status-current-style), never an exit code alone.
# Covers spec $9 table T1..T8.
set -uo pipefail

REPO_ROOT="$(cd "$(dirname "$0")/../../.." && pwd)"
WRAPPER="${WRAPPER:-$REPO_ROOT/scripts/tmx}"
HOST_OS="$(uname -s)"
case "$HOST_OS" in
    Darwin) TMUX_BIN_DEFAULT="$REPO_ROOT/tmux/build-darwin/bin/
             tmux" ;;
    *)      TMUX_BIN_DEFAULT="$REPO_ROOT/tmux/build/bin/tmux" ;;
esac
TMUX_BIN="${TMUX_BIN:-$TMUX_BIN_DEFAULT}"
STATE_BIN="$REPO_ROOT/scripts/tmx-state-bin"

echo "— Test 63: per-session color (operator-path) —"
PASS=0; FAIL=0; SKIP=0
_pass() { echo "PASS: $*"; PASS=$((PASS+1)); }
_fail() { echo "FAIL: $*"; FAIL=$((FAIL+1)); }
_skip() { echo "SKIP: $*"; SKIP=$((SKIP+1)); }

if [ ! -x "$TMUX_BIN" ]; then _skip "tmux binary not built"; echo
    "PASS=$PASS FAIL=$FAIL SKIP=$SKIP"; exit 0; fi
if [ ! -x "$STATE_BIN" ]; then _skip "tmx-state-bin not built";
    echo "PASS=$PASS FAIL=$FAIL SKIP=$SKIP"; exit 0; fi

# Isolated state file per run so persisted-color tests don't leak.
export TMX_STATE_FILE="/tmp/tmx_t63_state.$$"
trap 'rm -f "$TMX_STATE_FILE"; for s in "${CLEAN[@]}"; do
    "$WRAPPER" kill-session -t "$s" 2>/dev/null || true;
    "$TMUX_BIN" -L "tmx-$s" kill-server 2>/dev/null || true;
done' EXIT
CLEAN=()

# Helper: read a live global option value from a session's server.
```

```

_get_opt() { "$TMUX_BIN" -L "tmx-$1" show-options -gv "$2" 2>/dev/
null; }

# T1: name:color (named) → status-style bg=red
N="t63w1"; CLEAN+=("$N")
TMX_HOSTNAME= "$WRAPPER" new -s "$N:red" -d >/dev/null 2>&1
[ "$(_get_opt "$N" status-style)" = "bg=red" ] && _pass "T1
    name:red → bg=red" || _fail "T1 status-style='$_get_opt
    "$N" status-style'"

# T2: name:hex → status-style bg=#3b82f6
N="t63w2"; CLEAN+=("$N")
TMX_HOSTNAME= "$WRAPPER" new -s "$N:#3b82f6" -d >/dev/null 2>&1
[ "$(_get_opt "$N" status-style)" = "bg=#3b82f6" ] && _pass "T2
    name:hex → bg=#3b82f6" || _fail "T2 status-style='$
    ($_get_opt "$N" status-style)'"

# T3: all 4 surfaces reflect the color
N="t63w3"; CLEAN+=("$N")
TMX_HOSTNAME= "$WRAPPER" new -s "$N:blue" -d >/dev/null 2>&1
ok=1
[ "$(_get_opt "$N" status-style)"          = "bg=blue" ] ||
    ok=0
[ "$(_get_opt "$N" pane-active-border-style)" = "fg=blue" ] ||
    ok=0
[ "$(_get_opt "$N" clock-mode-colour)"      = "blue" ] ||
    ok=0
# window-status-current-style is bg=blue,fg=black
case "$(_get_opt "$N" window-status-current-style)" in
    bg=blue,fg=black) ;; *) ok=0;; esac
[ "$ok" = 1 ] && _pass "T3 all-4-surfaces blue" || _fail "T3
    surfaces mismatch"

# T4: persistence – set, kill, bare re-run reuses color
N="t63w4"; CLEAN+=("$N")
TMX_HOSTNAME= "$WRAPPER" new -s "$N:magenta" -d >/dev/null 2>&1
"$WRAPPER" kill-session -t "$N" 2>/dev/null || true
"$TMUX_BIN" -L "tmx-$N" kill-server 2>/dev/null || true
TMX_HOSTNAME= "$WRAPPER" new -s "$N" -d >/dev/null 2>&1 # BARE
    name
[ "$(_get_opt "$N" status-style)" = "bg=magenta" ] && _pass "T4
    persisted color wins on bare re-run" || _fail "T4 status-
    style='$_get_opt "$N" status-style'"

# T5: invalid color → non-zero exit AND no socket created
N="t63w5"
if TMX_HOSTNAME= "$WRAPPER" new -s "$N:notacolor" -d >/dev/null
    2>&1; then
    _fail "T5 invalid color accepted"
else
    if "$TMUX_BIN" -L "tmx-$N" ls >/dev/null 2>&1; then
        _fail "T5 invalid color created a server anyway";
        CLEAN+=("$N")
    else
        _pass "T5 invalid color rejected, no server created"
    fi
fi

```

```

fi
fi

# T6: escaped ':' in name → sanitised name, color applied
N="t63w6"; SAFE="t63w6" # 'a\:b' sanitises to a_b; use a value
                        whose safe form we know
RAW='t63w6\:x:cyan'      # name field "t63w6:x" → _sanitise →
                        "t63w6_x"
SAFE="t63w6_x"; CLEAN+=("$SAFE")
TMX_HOSTNAME= "$WRAPPER" new -s "$RAW" -d >/dev/null 2>&1
[ "$(_get_opt "$SAFE" status-style)" = "bg=cyan" ] && _pass "T6
    escaped colon → $SAFE bg=cyan" || _fail "T6 status-
    style='$_get_opt "$SAFE" status-style'"

# T7: extra fields ignored
N="t63w7"; CLEAN+=("$N")
TMX_HOSTNAME= "$WRAPPER" new -s "$N:green:x:y" -d >/dev/null 2>&1
[ "$(_get_opt "$N" status-style)" = "bg=green" ] && _pass "T7
    extra fields ignored → bg=green" || _fail "T7 status-
    style='$_get_opt "$N" status-style'"

# T8: hostname fallback for a fresh name (no persisted color) –
    status-style
# equals what hostname_color.sh would produce. We assert it is NOT
    empty and
# is a bg= token (the fallback path is intact).
N="t63w8"; CLEAN+=("$N")
TMX_HOSTNAME=fallbackhost "$WRAPPER" new -s "$N" -d >/dev/null
    2>&1
ss="$(_get_opt "$N" status-style)"
case "$ss" in bg=*) _pass "T8 hostname fallback → $ss" ;; *)
    _fail "T8 fallback status-style='$ss'" ;; esac

echo "— Test 63 result: PASS=$PASS FAIL=$FAIL SKIP=$SKIP —"
[ "$FAIL" -eq 0 ]

```

⚠ **Note for the implementer:** T6's escaped-colon example must match `_sanitise`'s actual output — `_sanitise` maps `:` → `_`, so the raw `t63w6\:x` produces name `t63w6:x` → sanitised `t63w6_x`. Verify by running `_sanitise` mentally / in a scratch shell before finalising the assertion. If the sanitised form differs, fix the `SAFE=` value, not the lib.



Step 2: Run the test

Run: `cd /Volumes/T7/Projects/tmux && bash scripts/tests/63_session_color.sh` Expected: PASS=8 FAIL=0 SKIP=0, exit 0.



Step 3: Determinism check (§11.4.50)

Run the test 3× and confirm identical PASS counts + identical readback values:

```

for i in 1 2 3; do bash scripts/tests/63_session_color.sh 2>&1 |
    tail -1; done

```


Expected: three identical PASS=8 FAIL=0 lines.



Step 4: Commit

```
cd /Volumes/T7/Projects/tmux
git add scripts/tests/63_session_color.sh
bash commit_all.sh "test($102): 63_session_color runtime operator-
    path suite (T1..T8, captured-evidence)"
```

Co-Authored-By: Claude Opus 4.8 (1M context)
<noreply@anthropic.com>

Task 8: Paired §1.1 mutations (M23 + M24) + pre-build gate

Files:

- Modify: scripts/tests/meta_test_false_positive_proof.sh

Interfaces:

- Produces: M23 (strip _apply_color → test 63 FAILs) + M24 (break set-color persistence → T4 FAILs), proving the tests catch the defect class.



Step 1: Add M23 + M24 to the meta-test harness

In scripts/tests/meta_test_false_positive_proof.sh, after the last existing run_mutation ... block, add (following the exact run_mutation <desc> <target_rel> <mutate_cmd> <revert_cmd> <test_rel> signature already used in the file):

```
# — M23: strip _apply_color from the wrapper → color tests FAIL
run_mutation \
    "M23: _apply_color stripped from wrapper" \
    "scripts/tmx.template" \
    "sed -i.tmp '/^_apply_color()/,/^\}/d/' scripts/tmx.template" \
    "mv scripts/tmx.template.tmp scripts/tmx.template" \
    "scripts/tests/63_session_color.sh" \
    "FAIL"

# — M24: break set-color persistence → T4 (persisted color) FAILs
run_mutation \
    "M24: set-color disabled in state binary" \
    "scripts/tmx-state/main.go" \
    "sed -i.tmp 's/func cmdSetColor/func cmdSetColor_DISABLED/' \
        scripts/tmx-state/main.go" \
    "mv scripts/tmx-state/main.go.tmp scripts/tmx-state/main.go" \
```

```
"scripts/tests/63_session_color.sh" \  
"FAIL"
```

Note for the implementer: `run_mutation` reverts via the `revert` cmd after asserting. For M24 the mutation renames the Go function, which causes a **build** failure (the dispatch case references `cmdSetColor`) — that build failure is itself the "test caught it" signal. Confirm against the harness's `expect_fail_regex` handling; if a build break doesn't match the FAIL regex cleanly, switch M24's `mutate` to instead make `cmdSetColor` return 1 immediately (persistence silently no-ops → T4 FAILs at runtime). Pick whichever the harness's exit-code path already rewards, and record the choice in the commit message (§11.4.6 no-guessing).



Step 2: Run the meta-test harness

Run: `cd /Volumes/T7/Projects/tmux && bash scripts/tests/meta_test_false_positive_proof.sh 2>&1 | grep -E 'M23|M24|Summary'`
Expected: M23 ... CAUGHT and M24 ... CAUGHT, and after revert the base tests PASS again.



Step 3: Commit

```
cd /Volumes/T7/Projects/tmux  
git add scripts/tests/meta_test_false_positive_proof.sh  
bash commit_all.sh "test($1.1): M23/M24 paired mutations for  
session-color (prove tests catch the defect)"
```

Co-Authored-By: Claude Opus 4.8 (1M context)
<noreply@anthropic.com>

Task 9: HelixQA Challenge + registration in `run_all`

Files:

- Modify: `scripts/challenges/tmux.yaml`
- Verify: `scripts/tests/run_all.sh` auto-discovers `[0-9][0-9]*.sh` (it does — no edit needed; just confirm).



Step 1: Add the Challenge

Append to `scripts/challenges/tmux.yaml` (next id after the existing max — find it via `grep id: scripts/challenges/tmux.yaml | tail`):

```
- id: TMUX-CH-XX          # replace XX with the next free  
  number  
  title: per-session color via name:color  
  description: |  
    tmx new -s NAME:COLOR creates a session whose status bar,  
    active-pane
```

```
border, clock, and current-window marker ALL render in COLOR
(validated:
tmux name / colour0-255 / #hex); the color persists and is
re-used on a
bare-name re-run; an invalid color is rejected with no
session created.
test_script: scripts/tests/63_session_color.sh
pass_condition: |
    test 63 prints "PASS=8 FAIL=0" AND a live show-options
    readback of
    status-style equals bg=<chosen color> for a colored session.
evidence: show-options readback transcript
severity: blocker
```



Step 2: Confirm run_all picks up tests 63 + 64

Run: `cd /Volumes/T7/Projects/tmux && grep -n '[0-9][0-9]_\' scripts/tests/run_all.sh` Expected: a glob loop that will auto-include `63_*.sh` and `64_*.sh` (no edit). If `run_all` maintains an explicit list instead, add both filenames there.



Step 3: Commit

```
cd /Volumes/T7/Projects/tmux
git add scripts/challenges/tmux.yaml
bash commit_all.sh "challenge($11.4.27): TMUX-CH-XX per-session
    color (captured-evidence)"
```

Co-Authored-By: Claude Opus 4.8 (1M context)
<noreply@anthropic.com>

Task 10: Docs + VERSION + CHANGELOG + release tag

Files:

- Modify: docs/guide/README.md, CLAUDE.md, Fixed.md, Issues.md, CHANGELOG.md, VERSION



Step 1: User guide subsection

In docs/guide/README.md, add a "Per-session color" subsection (after the session-management section) with the operator-path examples + the escape rule + the precedence rule (inline > persisted > hostname > green). Match the doc's existing tone.



Step 2: CLAUDE.md commands note

In CLAUDE.md, extend the tmx {new|attach|ls|kill} row note (or add a one-liner) noting the -s value may be name:color[:ignored].



Step 3: Tracker migration (Issues → Fixed)

Open then close a new ATM-NNN item in the same commit: add to Issues.md with Status Ready for testing, then migrate to Fixed.md with closure status per type (Fixed (→ Fixed.md) for Bug / Implemented (→ Fixed.md) for Feature) citing commit SHAs + the test-63 captured-evidence path. (Follow the repo's assign_atm_ticket_ids.sh for the id.)



Step 4: CHANGELOG + VERSION bump

Bump VERSION (version= + versionCode= together). Add a CHANGELOG.md entry: "Per-session color via tmx new -s name:color; persisted; #hex + tmux names; 4 surfaces; tmx-state v1.1.0 schema 2."



Step 5: Regenerate all doc exports (§11.4.65)

Run: cd /Volumes/T7/Projects/tmux && bash scripts/testing/sync_all_markdown_exports.sh 2>&1 | tail -5 Expected: every touched .md gets fresh .html/.pdf (and .docx where applicable) siblings.



Step 6: Full-suite retest (§11.4.40) before tag

Run: cd /Volumes/T7/Projects/tmux && nohup bash scripts/tests/run_all.sh > /tmp/tmx_runall_color.log 2>&1 & then poll the log until the summary appears. Expected: all tests GREEN (existing + 63 + 64). Any FAIL → §11.4.4 STOP + systematic-debug.



Step 7: Commit + tag

```
cd /Volumes/T7/Projects/tmux
git add docs/guide/README.md CLAUDE.md Fixed.md Issues.md
      CHANGELOG.md VERSION \
      $(find docs -name '*.html' -newer VERSION 2>/dev/null) \
      $(find docs -name '*.pdf' -newer VERSION 2>/dev/null)
bash commit_all.sh "release: per-session color
      (name:color[:params]) - docs + VERSION + changelog + ATM-
      NNN"
```

Co-Authored-By: Claude Opus 4.8 (1M context)
<noreply@anthropic.com>"

Tag prefixed per §11.4.151 (prefix from HELIX_RELEASE_PREFIX or
lowercased root dir).

PREFIX="\${HELIX_RELEASE_PREFIX:-tmux}"

```
git tag "${PREFIX}-<version>" # use the exact version string
                             from VERSION
bash commit_all.sh --sync-push "chore: push tag $
                             {PREFIX}-<version>" 2>/dev/null || git push github main --
                             tags && git push gitlab main --tags
```

Self-Review (run after writing; I did, fixes inline)

1. **Spec coverage:** spec §3 units → Tasks 1–6; §4 parsing → Task 5; §5 validation → Tasks 1+5 (cross-checked); §6 Go schema → Tasks 1–4; §7 wrapper → Task 6; §8 error handling → Task 6 (return 5); §9 testing → Tasks 7+8+9; §10 docs → Task 10; §11 parity → no-op (bridge unchanged, asserted in test 63's Darwin path via the same operator invocation); §13 acceptance → Task 10 step 6 full-suite + all earlier tasks. ✓
2. **Placeholder scan:** the only XX/<version> tokens are genuinely runtime-resolved (next free challenge id via grep; version via VERSION file) — flagged inline for the implementer, not stubs. ✓
3. **Type consistency:** `_parse_session_value/PARSED_NAME/PARSED_COLOR` used identically in Tasks 5+6; `_resolve_color/_apply_color/EFFECTIVE_COLOR` consistent across Tasks 6+7; Go `validColor/CanonColorNames` consistent across Tasks 1+2+3; `Session.Color` consistent across Tasks 2+3. ✓ (One typo `$sock_LABEL` was caught and corrected inline in Task 6 Step 1.)